

Lecture 4: Role-Based Access Control

CSC 1202: Software Development Project

Peter Wakholi (Ph.D)
SCIT Makerere University

AUTHENTICATION AND AUTHORISATION

Part 1



Introduction to Authentication and Authorization

- Authentication: Verifies user identity.
- Authorization: Determines user permissions.
- Together, they ensure secure access to systems.

Why Are Authentication and Authorization Important?



Prevents unauthorized access.



Protects sensitive data.



Ensures compliance with security standards.



Maintains system integrity and user accountability.

Types of Authentication

Single-factor authentication (SFA)

Two-factor authentication (2FA)

Multi-factor authentication (MFA)

Passwordless authentication

Common Authentication Methods

Username and Password

Biometric Authentication
(Fingerprint, Face ID)

One-Time Passwords (OTP)

Social Authentication
(Google, Facebook login)

Authorization Mechanisms

Role-Based Access
Control (RBAC)

Attribute-Based Access
Control (ABAC)

Discretionary Access
Control (DAC)

Mandatory Access
Control (MAC)

Role-Based Access Control (RBAC) Overview

Access is assigned
based on roles
(e.g., Admin, User,
Moderator).

Reduces
administrative
overhead.

Commonly used in
enterprise
applications.

RBAC vs. Other Access Control Models

- RBAC: Access based on roles.
- ABAC: Access based on attributes (e.g., location, device type).
- DAC: Users control access to their resources.
- MAC: Strict access policies enforced by administrators.

Implementing Authentication in Django

Django provides built-in authentication with User model.

Uses sessions, cookies, and tokens for authentication.

Third-party libraries like Django-Allauth can enhance authentication.

Implementing Authorization in Django

Django's permissions framework enables role-based access.

Use `@login_required` and `@permission_required` decorators.

Groups and user permissions can be managed via Django Admin.

Best Practices for Secure Authentication and Authorization

Use	Use strong password policies.
Implement	Implement multi-factor authentication (MFA).
Use	Use secure hashing algorithms (bcrypt, Argon2).
Implement	Implement session timeouts and account lockout mechanisms.

DESIGNING USER ROLES AND PERMISSIONS

Part 2

Introduction to User Roles and Permissions

- In modern applications, **user roles and permissions** are essential for managing access control and ensuring that users only have access to the data and actions relevant to their responsibilities.
- **User Roles** define a set of privileges assigned to users based on their function in the system.
- **Permissions** determine what actions a user can perform on specific resources.

Example in AITS:

In the **Academic Issue Tracking System (AITS)**, different users interact with the system in distinct ways:

- **Students** log issues related to academic records.
- **Lecturers/Heads of Departments** review and resolve assigned issues.
- **Academic Registrar** manages all issues and assigns them to relevant departments.
- **Administrators** configure the system and manage users.

Identifying User Roles in AITS

- To ensure security and efficiency, we define the following four key roles in AITS:

Role	Responsibilities	Permissions
Student	Logs academic issues, tracks issue status	Submit issue, View own issues
Lecturer	Reviews and resolves assigned issues	View assigned issues, Update issue status
Academic Registrar	Assigns issues to departments, resolves escalated issues	View all issues, Assign issues, Update issue status
Administrator	Manages users, system settings, and security policies	Manage users, Configure settings

illustration: User Roles in AITS

- A student logs a missing marks issue.
- The academic registrar assigns the issue to the responsible lecturer.
- The lecturer reviews the issue and updates the status.
- The student gets notified when the issue is resolved.
- The administrator ensures system integrity and manages role assignments.

Designing Role- Based Access Control (RBAC)

Step 1: Define System Resources

Resources in AITS include:

- **Issues** (logged by students, managed by staff)
- **Users** (students, lecturers, academic registrar, administrators)
- **Notifications** (alerts for status updates)
- **Audit Logs** (track all actions for security)

Step 2: Assign Permissions to Roles

Using Django's built-in permissions framework, we can define permissions such as:

- can_submit_issue
- can_assign_issue
- can_resolve_issue
- can_manage_users

```
from django.contrib.auth.models import Permission, Group
from django.contrib.contenttypes.models import ContentType
from myapp.models import Issue

# Create Student Group
student_group, created = Group.objects.get_or_create(name='Student')
permission = Permission.objects.get(codename='can_submit_issue')
student_group.permissions.add(permission)
```

Step 3: Implement Role-Based Views in Django

Django provides decorators to enforce permissions:

```
from django.contrib.auth.decorators import login_required, permission_required

@login_required
@permission_required('myapp.can_assign_issue', raise_exception=True)
def assign_issue(request):
    # Logic for assigning an issue
    return HttpResponse("Issue Assigned")
```

Managing User Roles and Permissions in AITS

- To manage roles effectively:
 - Assign Users to Roles: When a new user registers, they are assigned a default role.
 - Restrict Views and Actions: Use Django's `@permission_required` to restrict unauthorized actions.
 - Enable Role-Based Dashboards: Use React Router to show relevant dashboards based on the user role.
- Example: React Router Navigation for Role-Based Access

```
import { BrowserRouter as Router, Route, Redirect } from "react-router-dom";
import StudentDashboard from "../StudentDashboard";
import AdminDashboard from "../AdminDashboard";

<Router>
  <Route path="/student" component={StudentDashboard} />
  <Route path="/admin" component={AdminDashboard} />
  <Redirect from="/" to="/student" />
</Router>;
```

Best Practices for Designing User Roles and Permissions



Follow the Principle of Least Privilege (PoLP): Assign only the permissions users need.



Use Role-Based Dashboards: Provide a tailored experience based on user roles.



Implement Logging & Auditing: Track actions for security and troubleshooting.



Regularly Review Permissions: Update permissions as system needs evolve.

ASSIGNMENT: IMPLEMENTING USER AUTHENTICATION AND ROLE-BASED DASHBOARDS IN AITS

Part 3

Overview

- This assignment focuses on implementing user authentication and role-based dashboards in Django and React for the AITS. Students will use Django's authentication system to manage user logins and roles while setting up React Router to navigate between user dashboards based on their assigned roles.

Assignment Objectives

- By completing this assignment, students will:
 - Implement user registration and login in Django.
 - Use Django Groups and Permissions to manage roles.
 - Set up React Router to navigate to different dashboards based on user roles.
 - Restrict access to views and UI components based on user permissions.

1. Implement User Registration and Login (Django)

Create

Create a user registration form and store user credentials securely.

Implement

Implement login functionality using Django's authentication system.

Assign

Assign default roles (e.g., Student) to new users using Django Groups.

2. Set Up Role-Based Dashboards (React)

Create

Create React dashboards for different user roles (Student, Lecturer, Academic Registrar, Admin).

Use

Use React Router to navigate users to their respective dashboards based on their assigned role.

Implement

Implement UI restrictions so users only see actions permitted for their role.

React

React project files (App.js, StudentDashboard.js, AdminDashboard.js, Login.js).

3. Enforce Role-Based Access Control

Use Django's permissions and decorators to restrict access to views.

Implement middleware to prevent unauthorized access to protected routes.

Store user roles in localStorage to determine frontend navigation dynamically.

Django project files (views.py, urls.py, models.py, templates/).